

Prompt y Respuesta

Prompt:

Actúa como asistente de programación para una práctica de Estructura de Datos en Python.

Contexto:

Ya tengo un proyecto desarrollado en clase con Python, programación orientada a objetos y una interfaz gráfica creada con Qt Designer. En el proyecto ya existe una clase Stack con los métodos básicos de una pila. También existe una ventana o formulario donde se capturan datos y se muestran resultados, así como la clase que realiza la carga y la detección del evento para convertir la expresión.

Objetivo:

Necesito agregar al proyecto existente la funcionalidad para convertir una expresión infija a expresión posfija utilizando la estructura de pila ya creada en clase. Necesito se genere una clase que contenga una pila y un método para llevar a cabo la conversión.

Instrucciones importantes:

1. No generes un proyecto nuevo desde cero.
2. No reemplaces la clase Stack existente. Debes utilizarla e integrarte a su forma actual de trabajo.
3. Respeta la organización actual de archivos, clases, nombres de métodos y estilo de programación del proyecto.
4. Si necesitas proponer cambios, indícalos como cambios mínimos y explica exactamente en qué archivo y en qué parte deben colocarse.
5. La solución debe estar hecha con programación orientada a objetos.
6. La interfaz gráfica debe seguir usando el diseño generado con Qt Designer; no cambies el archivo .ui salvo que sea estrictamente necesario.
7. La aplicación debe permitir capturar una expresión infija, convertirla a posfija y mostrar el resultado en la interfaz.
8. Se asume que la expresión infija capturada es válida ya que de otra forma la conversión puede fallar.
9. Usa operadores básicos: +, -, *, /, \$(Potencia) y paréntesis.
10. Explica brevemente cómo funciona el algoritmo y cómo se utiliza la pila en la conversión, para esto genera un archivo .md que contenga esta explicación.

Código existente:

```
class Stack:
```

```
    def __init__(self):
```

```
        self.top = None
```

```
    def push(self, data):
```

```
        new_node = Node(data)
```

```
        if self.top is None:
```

```
            self.top = new_node
```

```
        else:
```

```
            new_node.next = self.top
```

```
            self.top = new_node
```

```
    def pop(self):
```

```
        if self.top == None:
```

```
            return "No hay elementos en la pila"
```

```
        else:
```

```
            temp = self.top.data
```

```
            self.top = self.top.next
```

```
            return f"El dato que se eliminó de la pila fue: {temp}"
```

```
    def top_of_stack(self):
```

```
        if self.top == None:
```

```
            return "No hay elementos en la pila"
```

```
        else:
```

```
            return f"El top de la pila es: {self.top.data}"
```

```
    def print_stack(self):
```

```
        self.x = "No hay elementos en la pila"
```

```
        if self.top == None:
```

```
            return self.x
```

```
        else:
```

```
            temp = self.top
```

```
            self.x = "top "
```

```

        while temp is not None:
            self.x += f"-> {temp.data} "
            temp = temp.next
        return self.x

    def is_empty(self):
        return self.top is None

import sys

from PyQt5.QtWidgets import QApplication
from load.load_menu import LoadMenu

def main():
    app = QApplication(sys.argv)
    menu = LoadMenu()
    menu.show()
    sys.exit(app.exec_())

if __name__ == "__main__":
    main()

from PyQt5.QtWidgets import QMainWindow
from PyQt5 import uic
from load.load_listas_enlazadas import DialogListasEnlazadas
from load.load_pila_stack import DialogStock

class LoadMenu(QMainWindow):
    def __init__(self):
        super().__init__()
        uic.loadUi("ui/menu.ui", self)
        self.actionListas_Enlazadas.triggered.connect(self.load_listas_enlazadas)
        self.actionPila_o_Stack.triggered.connect(self.loadPila_Stack)
        self.actionSalir.triggered.connect(self.close)

```

```

def load_listas_enlazadas(self):
    listas_enlazadas = DialogListasEnlazadas()
    listas_enlazadas.exec_()
def loadPila_Stack(self):
    pila_stack = DialogStock()
    pila_stack.exec_()
def close(self):
    return super().close()

```

Resultado esperado de tu respuesta:

- Primero analiza la estructura del código existente.
- Después propón la clase con el método para convertir de infija a posfija.
- Luego indica cómo conectar ese método con el botón o evento de la interfaz gráfica.
- Finalmente proporciona pruebas para comprobar que funciona.
- No omitas explicación, porque debo entender el código para defenderlo en clase ante el profesor y mis compañeros.

Respuesta:

1. Análisis de la estructura del código existente

Antes de agregar la conversión de infija a posfija, analicemos cómo está organizado tu proyecto.

Clase Stack

Ya tienes implementada una pila mediante una lista enlazada.

```

class Stack:
    def __init__(self):
        self.top = None

```

Internamente utiliza objetos Node.}

Los métodos disponibles son:

Método	Funcion
push(data)	Insertar un elemento al inicio de la pila
pop()	Elimina el elemento del tope
top_of_stack()	Devuelve el elemento superior
print_stack()	Imprime toda la pila
is_empty()	Indica si la pila está vacía

Problema que presenta la implementación actual

Los métodos

pop()

y

top_of_stack()

regresan cadenas de texto como:

"El dato que se eliminó de la pila fue: +"

Y

"El top de la pila es: *"

Para un algoritmo de conversión esto no es práctico, porque necesitamos obtener **únicamente el dato** (+, *, etc.).

No es necesario reemplazar la clase.

Basta con agregar **dos métodos nuevos**.

Cambio mínimo en Stack

En el archivo donde está definida la clase Stack, agrega al final:

```
def peek(self):  
    if self.top is None:  
        return None  
    return self.top.data
```

```

def pop_data(self):
    if self.top is None:
        return None

    temp = self.top.data
    self.top = self.top.next
    return temp

```

Observa que:

- NO eliminamos pop()
- NO modificamos top_of_stack()
- Solo agregamos dos métodos para poder reutilizar la pila dentro del algoritmo.

2. Nueva clase para convertir de Infija a Posfija

Crea un archivo nuevo, por ejemplo

estructuras/lineales/conversor_posfija.py

y coloca:

```
from estructuras.lineales.Pila_Stack import Stack
```

```
class ConversorPosfija:
```

```
    def __init__(self):
```

```
        self.pila = Stack()
```

```
    def prioridad(self, operador):
```

```
        prioridades = {
```

```
            '$': 3,
```

```
            '*': 2,
```

```
            '/': 2,
```

```
            '+': 1,
```

```
            '-': 1
```

```
        }
```

```
        return prioridades.get(operador, 0)
```

```

def convertir(self, expresion):
    salida = ""
    for caracter in expresion:
        if caracter.isalnum():
            salida += caracter
        elif caracter == "(":
            self.pila.push(caracter)
        elif caracter == ")":
            while (not self.pila.is_empty()) and self.pila.peek() != "(":
                salida += self.pila.pop_data()
            self.pila.pop_data()
        else:
            while (not self.pila.is_empty()) and self.pila.peek() != "(" and \
                self.prioridad(self.pila.peek()) >= self.prioridad(caracter):
                salida += self.pila.pop_data()
            self.pila.push(caracter)
    while not self.pila.is_empty():
        salida += self.pila.pop_data()
    return salida

```

3. Conectar con la interfaz

Supongamos que existe un botón

`self.btnConvertir`

y tu cajas de texto

`self.txtInfija`

Entonces únicamente agrega:

```
from estructuras.lineales.conversor_posfija import ConversorPosfija
```

En el constructor:

```
self.conversor = ConversorPosfija()
```

```
self.btnConvertir.clicked.connect(self.convertir)
```

Después crea el método:

```
def convertir(self):
```

```
    expresion = self.txt_infja.text()
```

```
    resultado = self.conversor.convertir(expresion)
```

```
    self.lbl_posfija.setText(resultado)
```

Eso es todo.

No es necesario modificar el archivo .ui, siempre que ya existan los controles.

Observación importante sobre el operador de potencia (\$)

El algoritmo anterior usa la condición:

```
while prioridad(pila.peek()) >= prioridad(caracter):
```

Esto funciona para +, -, * y /, pero **no para la potencia** si se desea el comportamiento matemático habitual. La potencia es un operador **asociativo por la derecha**